Snagboard: Campus Incident Reporting App

Technical Report, Formative Assignment 2 (Individual)

Wisdom Okechukwu

GitHub repository: https://github.com/owizdom/snagboard
Demo video: https://youtu.be/sGh7r2d4tlg

---

1. Introduction

Campus problems such as a dry tap, a dead socket or a broken chair are reported through group chats and hallway conversations. Students cannot tell whether anyone heard, and staff get the same complaint several times with no shared view of what is open.

Snagboard gives each problem (a “snag”) one record with one visible history. Students file a snag with an optional photo and follow it on a campus board. Staff move it along a fixed status track and must leave a note at every step.

2. Application Architecture

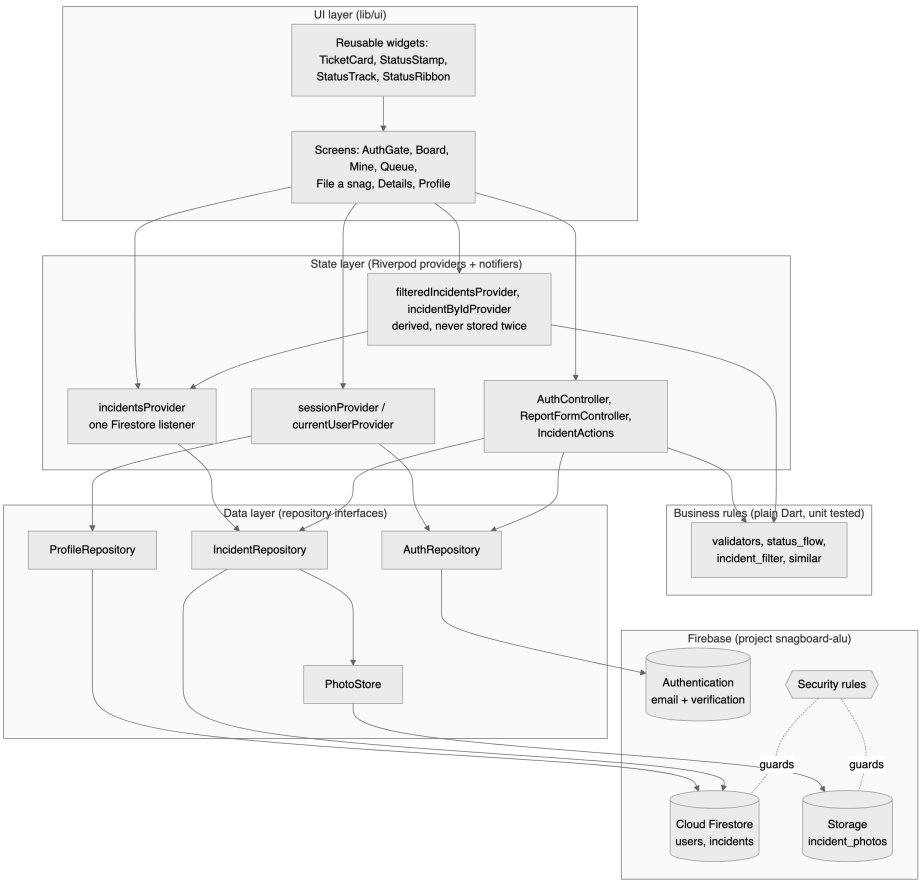


Figure 1. Architecture. Screens read Riverpod providers, providers call repository interfaces, and only the repositories know Firebase.

- `lib/ui`: the screens and shared widgets. No screen calls Firebase.
- `lib/state`: Riverpod providers and controllers (Section 4), plus the form checks.
- `lib/data`: the models and the repositories (`AuthRepository`, `ProfileRepository`, `IncidentRepository`, `PhotoStore`), the only code that talks to Firebase.

Navigation follows the authentication state (Figure 2). An `AuthGate` widget shows sign-in when signed out, a verify-email screen until the address is verified, and then the student or staff home.

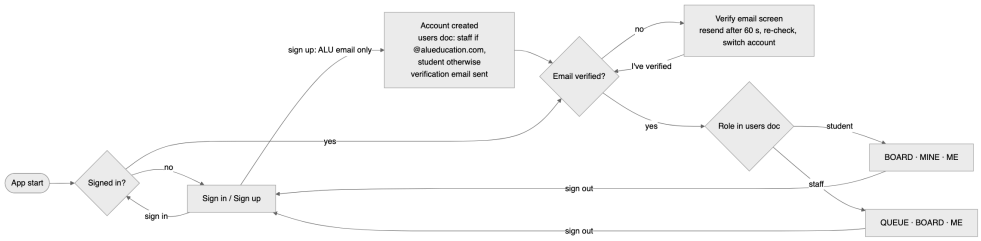


Figure 2. Authentication and navigation flow.

Where each Flutter concept lives in the code:

Concept	How the app uses it	Where
Widgets	Reusable pieces shared by every screen: `InkButton`, `InkField`, `ErrorText`, `TicketCard`, `StatusStamp`, `StatusTrack`	`ui/widgets.dart`, `ui/ticket.dart`
Navigation	`AuthGate` picks the first screen from the login state; `HomeShell` holds the tabs; `Navigator.push` opens filing and details	`ui/auth_screens.dart`, `ui/home_screens.dart`
Forms and validation	`Form` with validator functions (`validateAluEmail`, `validatePassword`, `validateTitle`); the filing controller checks each step before moving on	`state/report_form.dart`
Asynchronous work	Firestore streams feed `StreamProviders`; actions `await` Firebase calls; the photo upload reports progress before the Firestore write	`state/`, `data/incident_repository.dart`
State management	Riverpod: one `incidentsProvider` stream, derived providers for filters and details, `AsyncNotifier` controllers for actions	`state/`
Firebase	Auth for accounts and verification, Firestore for profiles and snags, Storage for photos, security rules on the server	`data/`, `firestore.rules`, `storage.rules`

3. Firebase Data Structure

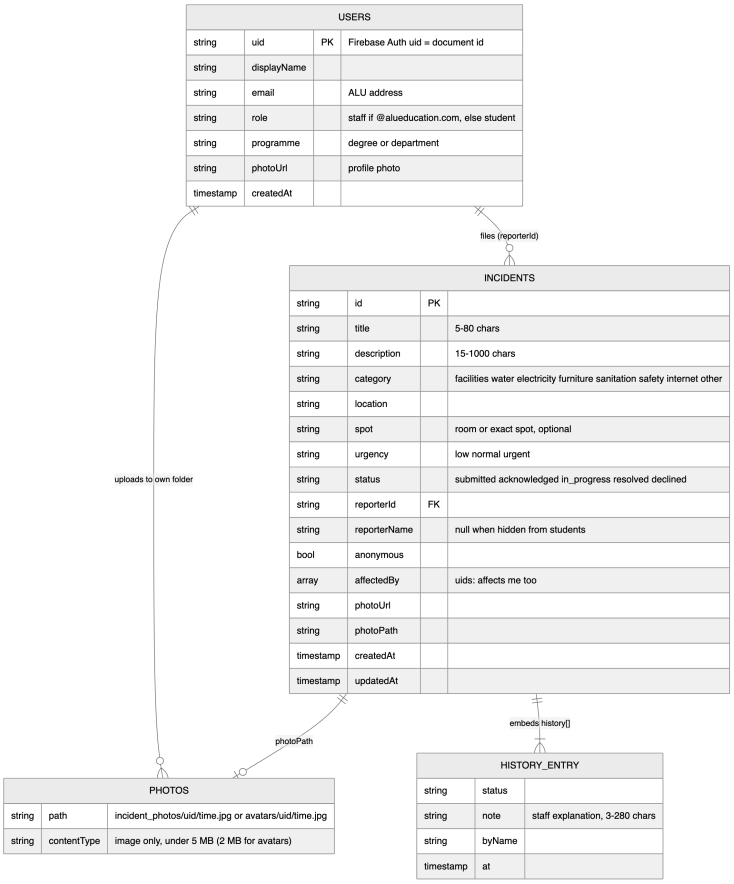


Figure 3. Firestore collections, the embedded status history, and Storage photos.

Location	One per	Main fields
`users/{uid}`	account	displayName, email, role (`student` or `staff`), programme (degree or department), photoUrl, createdAt
`incidents/{id}`	snag	title, description, category, location, spot, urgency, status, reporterId, reporterName, anonymous, affectedBy, history, photoUrl, photoPath, createdAt, updatedAt
Storage `incident_photos/{uid}/{time}.jpg`	snag photo	JPEG under 5 MB, resized to 1600 px
Storage `avatars/{uid}/{time}.jpg`	profile photo	image under 2 MB

Firestore organises documents into collections [2]. Each status change appends an entry (status, note, staff name, time) to the snag's `history` list. "Affects me too" is the `affectedBy` list of user ids. The app reads incidents with a single query: the 200 newest, ordered by creation time.

4. State Management Approach

The app uses Riverpod [5].

- Each Firebase stream is a `StreamProvider`, so every screen receives loading, error or data from one `AsyncValue` and shows a spinner or a retry panel the same way.
- One `incidentsProvider` holds the live incidents listener. The board, “Mine”, the staff queue and the details screen are derived from it, so a status change appears on every screen at once.
- Controllers run actions and expose their progress: `AuthController` (sign in, sign up, reset, verification), `ReportFormController` (the four-step draft, per-step validation, upload progress) and `IncidentActions` (status changes, “affects me too”, deletes).
- `setState` is used only for local widget details such as the 60-second resend countdown.

5. User Roles and Access Control

A verified `@alueducation.com` account becomes staff at sign-up and an `@alustudent.com` account becomes a student; the rules check the domain, so a student cannot claim staff. The project owner can still change any role in the Firebase Console. Firestore and Storage security rules [3] enforce access on the server. Every read and write requires a verified email [1] ending in `@alustudent.com` or `@alueducation.com`, except creating your own profile at sign-up, which happens before verification.

Action	Student	Staff
Read the board	Yes	Yes
File a snag	Yes	Yes
Edit or withdraw	Own, while submitted	No
Change status	No	Allowed transitions only, with a note
“Affects me too”	Own id only	Own id only
Remove a snag	Own, while submitted	Any
Read another user’s profile	No	Yes
Edit own name, degree or department, photo	Yes	Yes
Change a role	No	No
Upload a photo	Own folder, image under 5 MB	Same

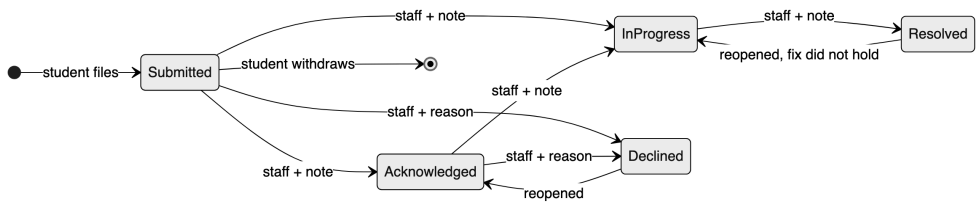
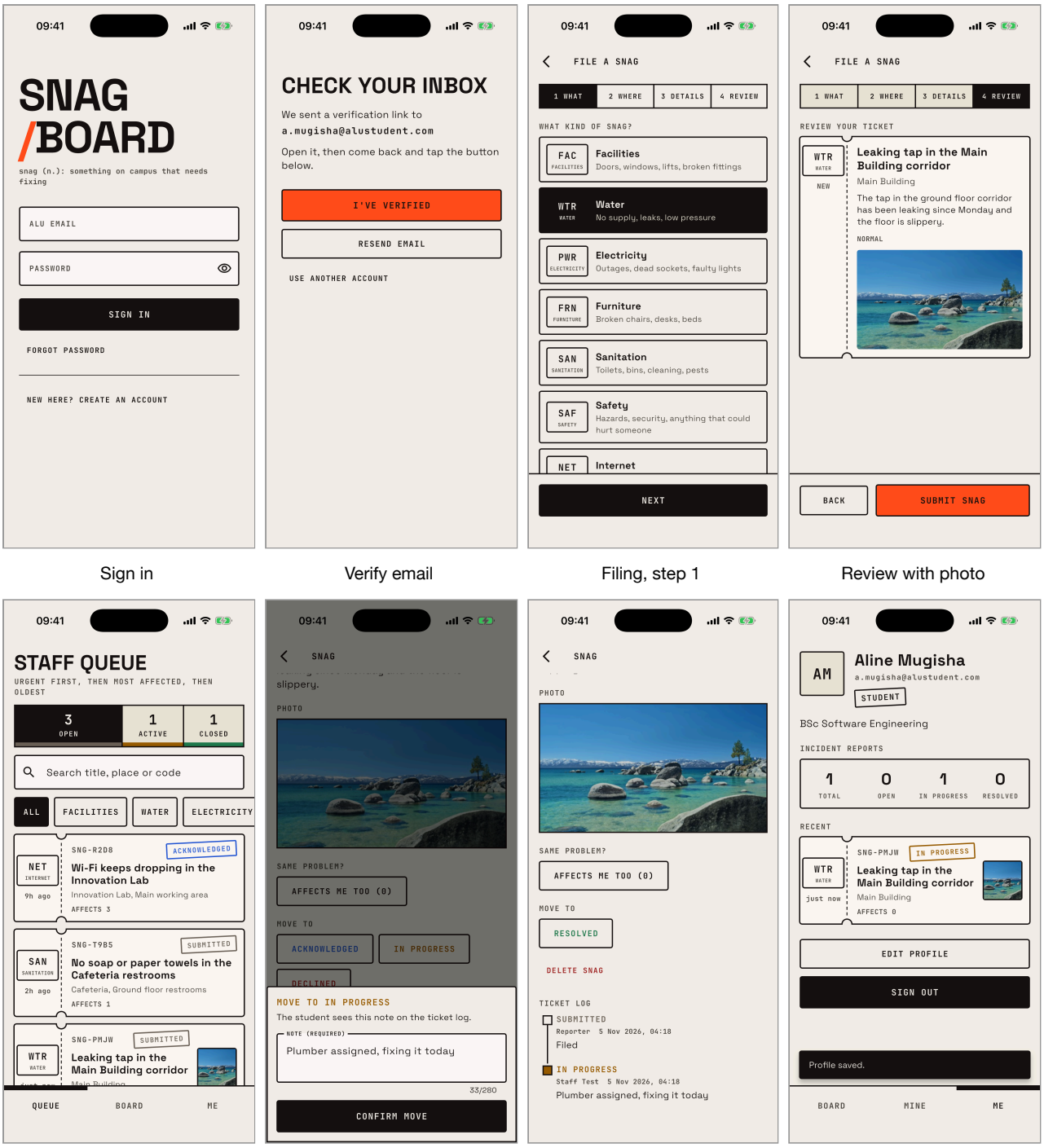


Figure 4. Status flow. Each staff move appends a history entry with a note.

Privacy and misuse:

- A student can hide their name from other students; staff still see it.
- Before filing, the app lists open snags with the same category and location and offers “Affects me too” instead of a duplicate.
- Choosing Safety shows that the app is not monitored around the clock and points to campus security.
- Students cannot read each other’s profiles or emails.

6. UI/UX Decisions



Sign in | Verify email | Filing, step 1 | Review with photo

Staff queue | Status change with note | Ticket log | Profile

Figure 5. Main workflows on the iPhone 17 Pro simulator: authentication, filing a snag, staff triage, status tracking and the profile.

The look is a work-order board: paper background, black ink, one orange accent, 2 px outlines, and monospaced ticket codes with stamped statuses. All text colours meet the 4.5:1 contrast ratio [8].

Decision	Reason
Filing in four steps (What, Where, Details, Review)	One question per screen, errors shown beside the field [7]
Categories as rows with examples	Students choose by meaning, not by icon
Status ribbon split into open, active and closed	Campus overview at a glance; each segment filters
Vertical status track with names and notes	The latest note answers “what is happening?”
Profile with photo, degree, report summary and recent snags	Students see their own record at a glance
Loading, empty and error states on every screen, errors offer Retry and Sign out	The user always knows what the app is doing and is never stuck

7. Testing Strategy

Tests follow Flutter’s testing guide [6] for app logic and Firebase’s rules testing guide [9] for the server rules.

Level	Covers	Tests
App logic	ALU email check, password and title rules, allowed status moves	4
Security rules	Staff sign-up by domain, a student cannot claim staff, verified ALU reads only, filing, staff status moves with a note, a student cannot change a status; run on the Firestore emulator [9]	9

The final run passed 4 of 4 app tests and 9 of 9 rules tests. Each main workflow was also checked by hand on the iPhone 17 Pro simulator against the live Firebase project, as recorded in the demo video:

Manual check	Result
Sign up with an ALU address and verify from the inbox	Account listed in Authentication, app opens
File a snag with a photo	Snag in Firestore with its first history entry, photo shown on the card
Filter by category	Water shows the snag; Electricity shows “No matches”
Staff moves the snag to In progress with a note	Note appended to the history in Firestore
Sign out as staff, sign in as the student	Student sees In progress and the staff note

8. Challenges Encountered

- Firebase Storage now requires the pay-as-you-go Blaze plan [4]. The project was upgraded and the bucket placed in a no-cost region.
- Riverpod 3 retries failed providers automatically, which left an endless spinner; retries are off so errors show a Retry button.
- Sign-up writes the profile before the email is verified, and the rules refused it. Profile creation is now the one write allowed before verification.
- After verifying, Firebase keeps the old login token, which still says “not verified”, so the rules kept refusing the user. The app now refreshes the token before moving on.

9. Lessons Learned

- Security belongs in the rules, and the rules need their own tests, including the refused cases.
- Deriving every screen from one stream removes “updated here but not there” bugs.
- A live listener must restart for each signed-in user; otherwise the refusal of the last account’s listener shows up as an error for the next one.

10. Limitations and Future Work

- Search runs on the phone over the 200 newest snags; a larger campus needs server-side search.
- No server-side rate limit on filing; App Check and a Cloud Function quota would add one.
- No push notifications; Firebase Cloud Messaging on status change is the next step.
- A hidden-name snag still carries the reporter’s user id, so two snags by the same student can be linked.
- Photos of removed snags stay in Storage until a cleanup job exists.
- Every `@alueducation.com` account gets the same staff powers; roles per department are future work.
- Tested on iOS only.

11. Major Technical Decisions

Decision	Alternative	Why
Riverpod	Provider, BLoC	Streams map directly to providers; derived providers keep screens in sync without a BuildContext; screens share data without passing it down
Repositories	Firebase calls in widgets	All Firebase code sits in two files in `lib/data`
History embedded in the snag	`updates` subcollection	One read for details; a status change and its note are one atomic write the rules can check
One query, filters on the phone	A query per filter	No composite indexes, one listener, fine at campus scale
Rules enforce status transitions	Trust the app	A modified app cannot skip steps or rewrite history
Role from the verified email domain, editable in the Console	Admin-only appointment	Graders and staff are staff on sign-up with no manual step; the owner can still correct a role

References

[1] Google, “Manage users in Firebase (send a user a verification email),” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/auth/flutter/manage-users. Accessed: Nov. 5, 2026.

[2] Google, “Cloud Firestore data model,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/firestore/data-model. Accessed: Nov. 5, 2026.

[3] Google, “Get started with Cloud Firestore Security Rules,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/firestore/security/get-started. Accessed: Nov. 5, 2026.

[4] Google, “FAQs about changes to Cloud Storage for Firebase pricing and default buckets,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/storage/faqs-storage-changes-announced-sept-2024. Accessed: Nov. 5, 2026.

[5] R. Rousselet, “Getting started,” Riverpod Documentation. [Online]. Available: https://riverpod.dev/docs/introduction/getting_started. Accessed: Nov. 5, 2026.

[6] Google, “Testing Flutter apps,” Flutter Documentation. [Online]. Available: https://docs.flutter.dev/testing/overview. Accessed: Nov. 5, 2026.

[7] J. Nielsen, “10 usability heuristics for user interface design,” Nielsen Norman Group, 1994, updated Jan. 30, 2024. [Online]. Available: https://www.nngroup.com/articles/ten-usability-heuristics/. Accessed: Nov. 5, 2026.

[8] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, Jun. 5, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/. Accessed: Nov. 5, 2026.

[9] Google, “Test your Cloud Firestore Security Rules,” Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/rules/unit-tests. Accessed: Nov. 5, 2026.